

Login con Google en React Native: por qué funcionaba en todos lados menos donde importaba

Un problema real de autenticación en React Native, diagnosticado y resuelto sobre dispositivo físico.

Fernando Fernández González

Full Stack Developer · León, España

[linkedin.com/in/fernandofdezglez](https://www.linkedin.com/in/fernandofdezglez)

Contexto

Una app **React Native (Expo)** con autenticación vía Supabase necesitaba login con Google. El backend de esta app vive dentro de un **monorepo Turborepo/pnpm**, expuesto como una API **Hono** montada sobre funciones serverless de **Vercel**. Ya existía una implementación de referencia funcionando en otro proyecto con un stack de backend distinto (persistente, no serverless), así que la expectativa era portar esa solución sin demasiados sobresaltos.

No fue así. El flujo funcionaba perfectamente en emulador y en development build. En el APK real de producción, instalado en un dispositivo físico, el login parecía completarse y la app devolvía al usuario a la pantalla de inicio sin ningún mensaje de error visible.

Cada ciclo de prueba requería generar un nuevo build en la nube, con un tiempo de espera considerable y sin logs claros que indicaran dónde estaba el fallo. Diagnosticar esto a ciegas, build tras build, no era viable — y la combinación de piezas en juego (monorepo, serverless, framework de API ligero, React Native) multiplicaba los sitios donde algo se podía estar perdiendo por el camino.

Por qué la solución de referencia no era portable

La implementación que funcionaba en el otro proyecto se apoyaba en dos condiciones que no se cumplían aquí:

1. **Backend persistente vs. serverless dentro de un monorepo.** El proyecto de referencia corría sobre un backend Node.js persistente, sin restricciones de CDN. Este proyecto, en cambio, expone su API (Hono) como funciones serverless de Vercel dentro de un monorepo Turborepo/pnpm — una capa más de indirección entre el cliente y el código que realmente procesa la petición. En esa combinación concreta, el header `Authorization: Bearer` no llegaba de forma consistente a la función backend. La solución fue usar un header custom (`X-Auth-Token`) que no atravesaba esa capa intermedia.

Esto no es una limitación universal de los entornos serverless — es específico a la combinación de framework de API, adaptador HTTP y capa

de enrutamiento usada en este caso. No debería generalizarse sin verificar el stack concreto.

2. **Redirect incompatible con el flujo OAuth estándar (PKCE).** Con el flow recomendado por Supabase, el redirect final ignoraba el deep link de la app y volvía a la URL web configurada como "Site URL". Confirmado revisando los logs de Supabase: el callback llegaba con status 302 apuntando al dominio web, no al scheme custom de la app.

La solución fue cambiar el flujo a `implicit` y ajustar el "Site URL" al scheme propio de la app (`app://`). Es un workaround operativo, no la arquitectura ideal — el flujo PKCE sigue siendo el estándar correcto y quedó como mejora pendiente.

3. **El método recomendado para abrir el navegador de autenticación no disparaba el deep link en Android.** El método que usa pestañas vinculadas al proceso de la app abría y cerraba la pantalla de login sin que React Native recibiera ningún evento. Confirmado con `adb logcat` en dispositivo físico: la actividad de navegador se abría y cerraba, pero ningún evento de deep link llegaba a la capa JS.

El objetivo era que el usuario apenas percibiera el salto: un instante de navegador para el login de Google, y vuelta automática a la app ya autenticado, sin fricción. Conseguir esa transición de forma fiable en Android resultó mucho más complicado de lo esperado. La solución que finalmente funcionó fue abrir el navegador completo del sistema (no una pestaña vinculada al proceso), de modo que el sistema operativo pudiera reconocer el scheme de la app vía sus intent filters y redirigir correctamente al volver. El resultado final es el que se buscaba desde el principio — el usuario ve el navegador un instante y vuelve logueado — pero llegar hasta ahí de forma estable costó bastante más de lo previsto.

4. **El método que Supabase ofrece para extraer la sesión de una URL no existe en React Native** (depende de `window.location`, que no existe fuera del navegador). Fue necesario parsear manualmente el fragmento de la URL de retorno (`#access_token=...`) siguiendo el formato estándar OAuth2 (RFC 6749).

El síntoma final — y por qué tardó varias jornadas en aparecer

Con las cuatro piezas anteriores resueltas, el login funcionaba... en desarrollo. En producción, seguía sin funcionar, y esta vez sin ninguna pista visible.

La sesión de un desarrollador terminó con `adb logcat` conectado al dispositivo físico, filtrando únicamente los logs de la capa JavaScript. Ahí apareció la primera pista real: **el token de acceso llegaba perfectamente**. El login, en sí, funcionaba.

El problema estaba en lo que pasaba justo después. Cruzando esos logs con los del backend, apareció la causa: cuando un usuario nuevo (sin cuenta previa en la plataforma) iniciaba sesión por primera vez, el backend respondía correctamente con un error 404 ("usuario no encontrado") — la señal esperada para redirigir a onboarding. Pero el mensaje de error que llegaba al cliente no incluía el código de estado HTTP, solo el texto. La lógica que debía detectar "usuario nuevo → onboarding" buscaba ese código en el mensaje, no lo encontraba, y por defecto cerraba la sesión, devolviendo al usuario a la pantalla de inicio.

El fix fue de una línea: incluir el código de estado HTTP en el mensaje de error lanzado por el cliente API, en vez de solo el texto descriptivo.

Por qué fallaba en producción pero no en desarrollo

El comportamiento del ciclo de vida de las actividades de Android difiere entre un development build y un APK de producción con `launchMode="singleTask"` — el intent que transporta el deep link se entrega de forma distinta y con un timing distinto, exponiendo una condición de carrera que en desarrollo pasaba desapercibida.

Lecciones

- Los logs del dispositivo físico (`adb logcat`) fueron más útiles que cualquier documentación oficial para aislar el problema real.

- El comportamiento de Android puede variar según el dispositivo y la capa de personalización del fabricante — validar siempre en más de un dispositivo físico antes de dar un fix por bueno.
- Las cabeceras HTTP pueden perderse en infraestructura intermedia sin que sea evidente por qué — diagnosticar con logs de red antes de asumir que el problema está en el código de la app.
- Ningún método de apertura de navegador es universalmente compatible en Android — conviene comprobar el comportamiento real en dispositivo, no solo en emulador.
- Un guard de navegación (login → onboarding → app) necesita modelar explícitamente sus estados transitorios, no solo los estados finales.
- El mensaje de error debe llevar siempre el código de estado HTTP explícito — depender solo del texto para la lógica de control es frágil.
- El comportamiento entre development build y APK de producción puede diferir de forma sutil pero decisiva — nunca dar un fix por validado sin probarlo en un build de producción real.

Deuda técnica reconocida

Esta solución es pragmática, no ideal. Quedan pendientes, por orden de prioridad:

- Migrar el almacenamiento de tokens a un almacén cifrado a nivel de sistema operativo (en vez de almacenamiento simple).
- Añadir protección contra doble procesamiento del callback de autenticación.
- Cubrir casos de error del flujo OAuth no contemplados (cancelación del usuario, timeout).
- Migrar de vuelta a PKCE en cuanto el proveedor de autenticación resuelva el comportamiento de redirect en mobile.
- Sustituir el scheme custom por App Links / Universal Links, que ofrecen verificación de propiedad del dominio.

Caso de estudio basado en un problema real de producción, diagnosticado y resuelto sobre dispositivo físico Android.